



**UNIWERSYTETY
WSB MERITO**

DOKUMENTACJA PROJEKTU

ResourcePilot

Aplikacja webowa do rezerwacji stolików w restauracji

Przedmiot:	Programowanie Obiektowe
Rok akademicki:	2025 / 2026
Kierunek / Grupa:	Informatyka, 2CW I INF N 2
Prowadzący:	Mgr inż Arkadiusz Banasik
Data oddania:	14.06.2026r.

SKŁAD ZESPOŁU

Imię i Nazwisko	Nr indeksu	Rola w projekcie
Marcin Harazmus	183601	Frontend (React / TypeScript)
Daniel Kokot	183615	API & Domain (.NET / C#)
Maciej Hudek	189288	Baza danych, Integracja, Testy i Dokumentacja

1. Opis projektu

ResourcePilot to aplikacja webowa umożliwiająca rezerwację stolików w restauracji. Projekt został zrealizowany w ramach przedmiotu Programowanie Obiektowe i stanowi pełnoprawny system składający się z backendu REST API, bazy danych PostgreSQL oraz responsywnego frontendu w technologii React.

1.1 Cel projektu

Celem projektu było zaprojektowanie i implementacja systemu rezerwacji stolików z podziałem na warstwę klienta (frontend) oraz warstwę serwera (backend), z zastosowaniem wzorców projektowych charakterystycznych dla programowania obiektowego.

1.2 Funkcjonalności systemu

- Przeglądanie dostępnych stolików w czasie rzeczywistym
- Rezerwacja stolika z wyborem daty, godziny i liczby gości
- Zarządzanie rezerwacjami przez gościa (podgląd, anulowanie)
- Panel administracyjny restauracji (lista rezerwacji, zmiana statusu)
- Walidacja danych wejściowych po stronie frontendu i backendu
- Obsługa kolizji rezerwacji — blokada stolika na 2 godziny

2. Architektura systemu

Projekt oparty jest o wzorzec Clean Architecture, który dzieli aplikację na niezależne warstwy z wyraźnie określonymi odpowiedzialnościami. Każda warstwa może zależeć tylko od warstw leżących głębiej — nigdy od warstw wyższych.

2.1 Warstwy backendu

Projekt	Odpowiedzialność	Zawiera
ResourcePilot.Domain	Encje i enumeracje dziedziny	Table, Reservation, ReservationStatus
ResourcePilot.Application	Interfejsy i obiekty transferu danych	IReservationService, DTOs
ResourcePilot.Infrastructure	Implementacja dostępu do bazy danych	DbContext, DbSeeder, Migracje EF Core
ResourcePilot.Api	Kontrolery HTTP i serwisy	Controllers, ReservationService, Program.cs

2.2 Stack technologiczny

Warstwa	Technologia	Wersja
Backend	ASP.NET Core Web API (C#)	.NET 8 / 10
ORM	Entity Framework Core + Npgsql	8.x / 10.x
Baza danych	PostgreSQL	16 / 18
Frontend	React + TypeScript (Vite)	18.x
Stylowanie	Tailwind CSS + Framer Motion	-
Dokumentacja API	Swagger / OpenAPI	3.0

3. Model danych

Baza danych PostgreSQL zawiera dwie tabele główne powiązane relacją jeden-do-wielu. Schemat został wygenerowany automatycznie przez Entity Framework Core na podstawie klas dziedziny zdefiniowanych w projekcie ResourcePilot.Domain.

3.1 Tabela Tables (stoliki)

Kolumna	Typ	Opis
Id	integer (PK)	Unikalny identyfikator stolika
Number	integer	Numer stolika widoczny dla klienta
Capacity	integer	Maksymalna liczba miejsc
Description	text	Opis lokalizacji, np. 'Przy oknie'
IsActive	boolean	Czy stół jest aktywny (soft-disable)

3.2 Tabela Reservations (rezerwacje)

Kolumna	Typ	Opis
Id	integer (PK)	Unikalny identyfikator rezerwacji
TableId	integer (FK)	Klucz obcy do tabeli Tables
CustomerName	text	Imię i nazwisko klienta
Email	text	Adres email klienta
Phone	text	Numer telefonu klienta
ReservationDate	date	Data rezerwacji
ReservationTime	time	Godzina rozpoczęcia rezerwacji
GuestCount	integer	Liczba gości
Notes	text	Opcjonalne uwagi klienta
Status	integer (enum)	0=Pending, 1=Confirmed, 2=Cancelled
CreatedAt	timestampz	Data i czas złożenia rezerwacji (UTC)

4. Dokumentacja API (REST)

Wszystkie endpointy dostępne są pod bazowym adresem `http://localhost:{PORT}/api`.
Interaktywna dokumentacja dostępna jest pod adresem `/swagger` w środowisku deweloperskim.

4.1 Stoliki — `/api/Tables`

Metoda	Endpoint	Opis	Odpowiedź
GET	<code>/api/Tables/available?date=&time=&guestCount=</code>	Zwraca wolne stoliki dla podanego terminu	200 OK

4.2 Rezerwacje — `/api/Reservation`

Metoda	Endpoint	Opis	Odpowiedź
GET	<code>/api/Reservation</code>	Lista wszystkich rezerwacji (admin)	200 OK
GET	<code>/api/Reservation/{id}</code>	Szczegóły rezerwacji po ID	200 / 404
POST	<code>/api/Reservation</code>	Tworzenie nowej rezerwacji (formularz)	201 Created
PATCH	<code>/api/Reservation/{id}</code>	Aktualizacja statusu lub notatki (admin)	200 / 404
DELETE	<code>/api/Reservation/{id}</code>	Anulowanie rezerwacji (soft delete)	204 / 404

5. Zastosowane wzorce projektowe

W projekcie zastosowano szereg wzorców projektowych zgodnych z zasadami programowania obiektowego (SOLID) oraz dobrymi praktykami wytwarzania oprogramowania.

5.1 Dependency Injection

Serwisy rejestrowane są w kontenerze IoC frameworka ASP.NET Core. Kontrolery i serwisy nie tworzą własnych zależności — otrzymują je z zewnątrz przez konstruktor, co umożliwia łatwe testowanie i podmianę implementacji.

5.2 Repository Pattern (przez EF Core)

Entity Framework Core pełni rolę warstwy dostępu do danych. DbContext hermetyzuje wszystkie operacje na bazie danych, oddzielając logikę biznesową od szczegółów implementacji zapytań SQL.

5.3 DTO (Data Transfer Object)

Dane przesyłane między frontendem a backendem są reprezentowane przez dedykowane klasy DTO (np. CreateReservationDto, ReservationResponseDto), które oddzielają model domenowy od kontraktu API.

5.4 Service Layer

Logika biznesowa (walidacja dostępności stolika, sprawdzanie kolizji terminów) jest wydzielona do klasy ReservationService implementującej interfejs IReservationService, zgodnie z zasadą pojedynczej odpowiedzialności (SRP).

5.5 Adapter Pattern (frontend)

Plik reservationApi.ts pełni rolę adaptera między formatem danych frontendu a kontraktem REST API backendu, tłumacząc m.in. różnice w nazewnictwie pól i formatach dat.

6. Instrukcja uruchomienia projektu

6.1 Wymagania

- PostgreSQL 16+ (z działającym użytkownikiem admin i bazą restaurant)
- .NET SDK 8.0 lub 10.0
- Node.js 18+ oraz npm
- dotnet-ef: dotnet tool install --global dotnet-ef

6.2 Backend

Wszystkie komendy wykonywane z katalogu głównego repozytorium:

Krok	Komenda
1	dotnet ef database update --project ResourcePilot.Infrastructure --startup-project ResourcePilot.Api
2	dotnet run --project ResourcePilot.Api
3	Otwórz przeglądarkę: http://localhost:{PORT}/swagger

6.3 Frontend

Krok	Komenda
1	cd ResourcePilot.Ui
2	npm install
3	npm run dev → http://localhost:5173

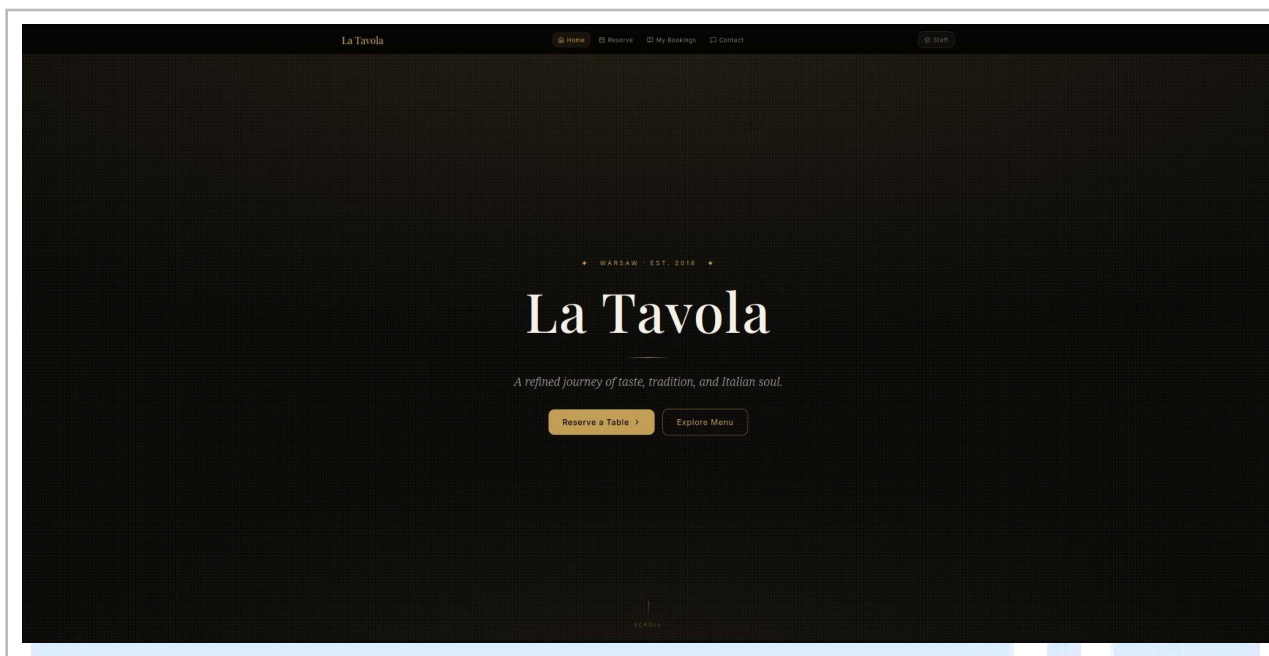
Uwaga: Należy upewnić się że port backendu w pliku `src/shared/reservationApi.ts` (stała `API_BASE`) zgadza się z portem na którym nasłuchuje backend (widoczny po uruchomieniu `dotnet run`).

7. Zrzuty ekranu z działania aplikacji

Poniżej przedstawiono zrzuty ekranu prezentujące kluczowe funkcjonalności systemu.

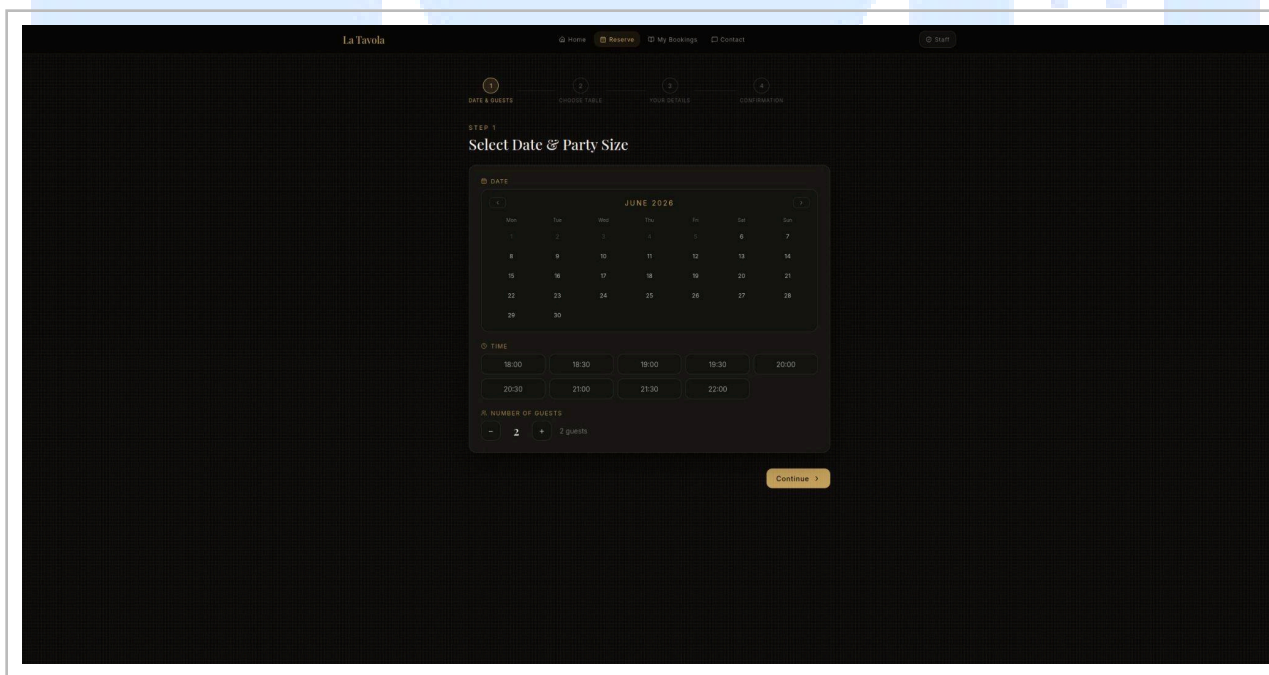
7.1 Strona główna (Landing Page)

Widok strony głównej restauracji z możliwością przejścia do formularza rezerwacji.



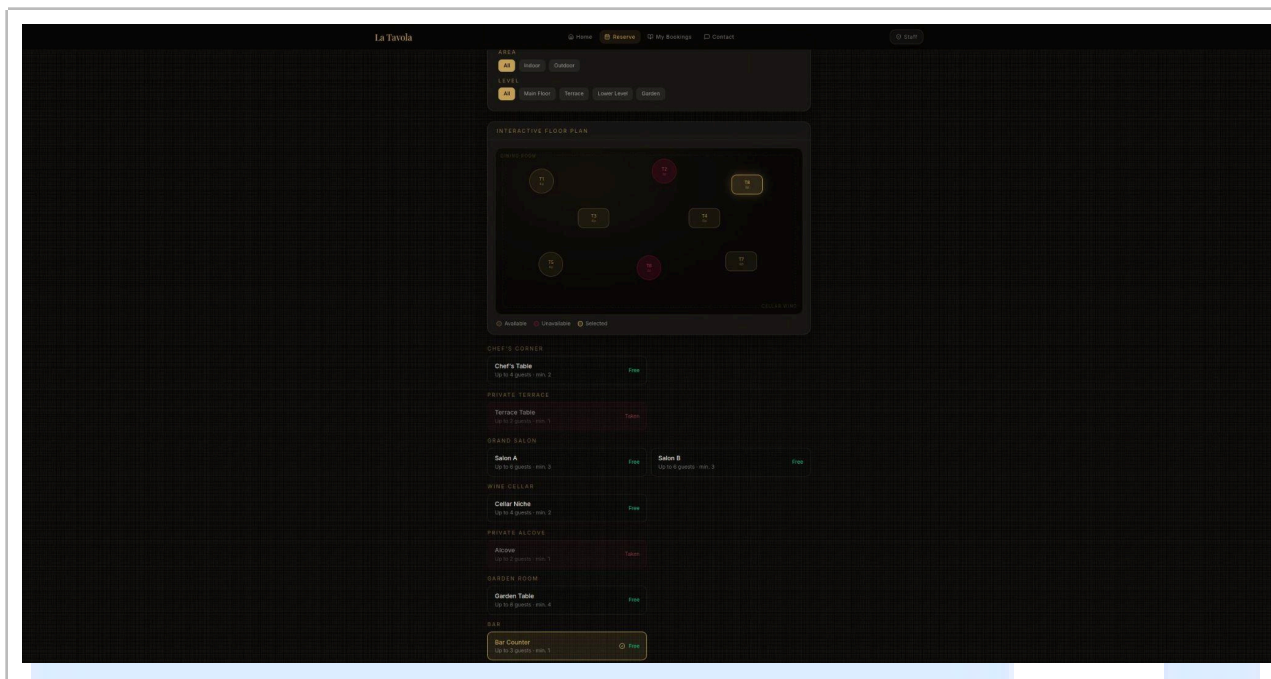
7.2 Formularz rezerwacji — Krok 1

Wybór daty, godziny i liczby gości.



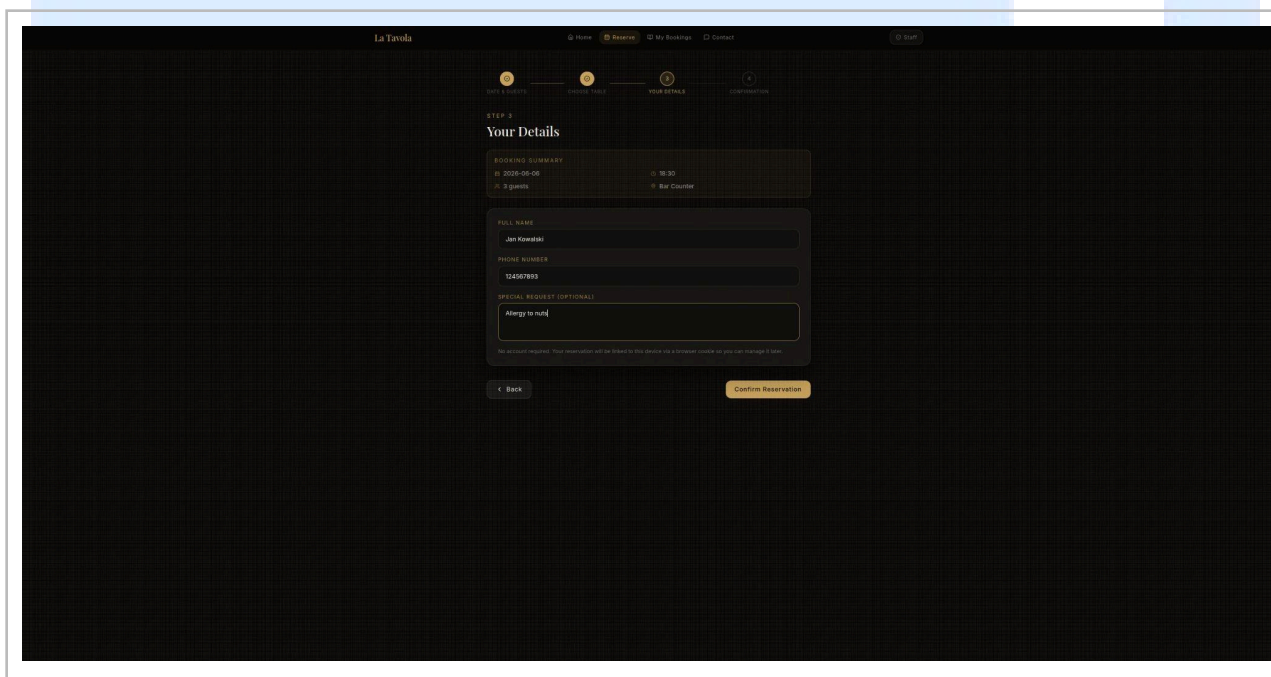
7.3 Formularz rezerwacji — Krok 2

Wybór stolika na planie sali na podstawie dostępności.



7.4 Formularz rezerwacji — Krok 3

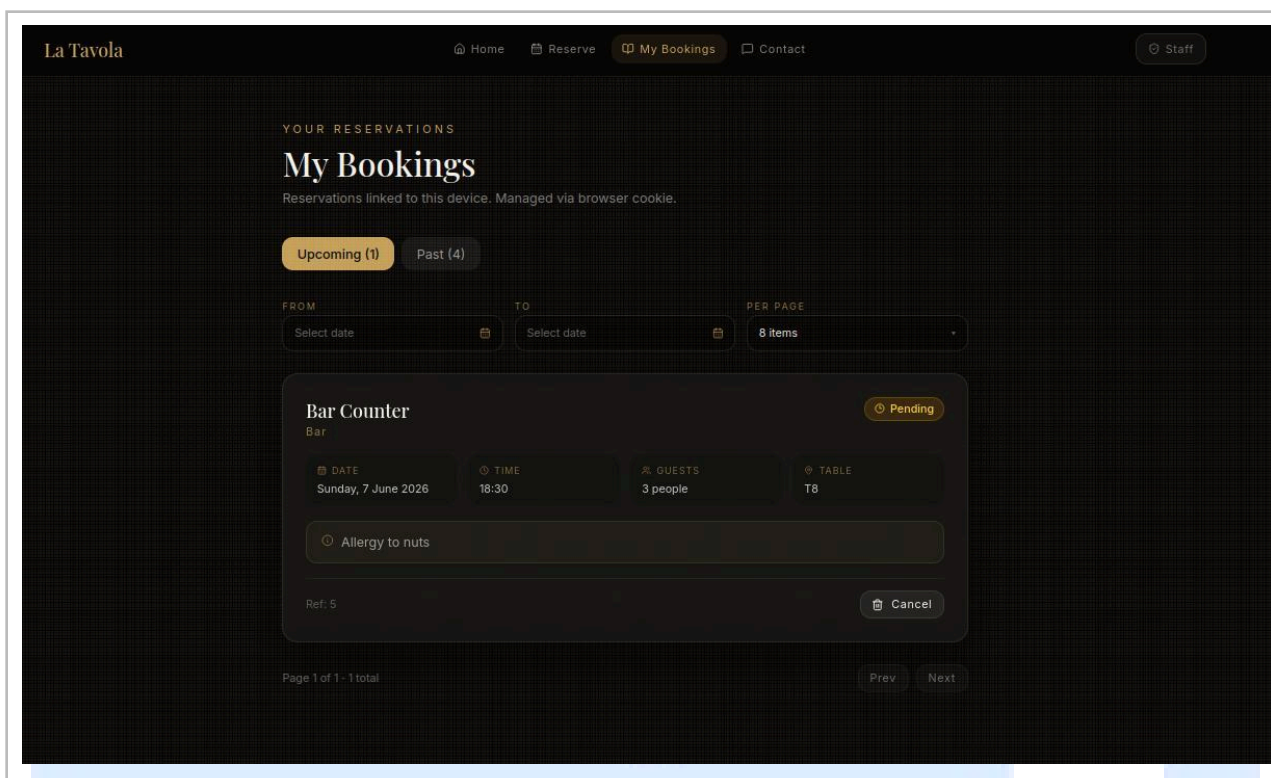
Wprowadzenie danych osobowych gościa.



7.5 Potwierdzenie rezerwacji

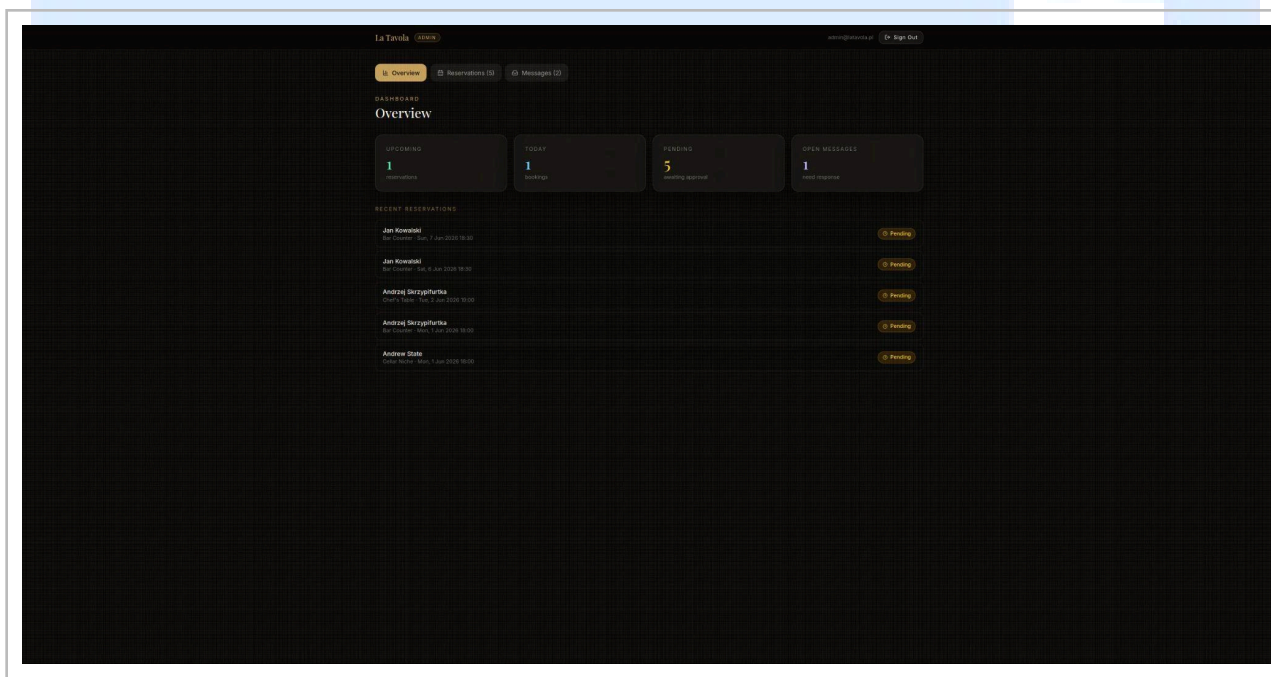
Ekran potwierdzenia po pomyślnym złożeniu rezerwacji.

ResourcePilot_dokumentacja



7.6 Panel administracyjny — lista

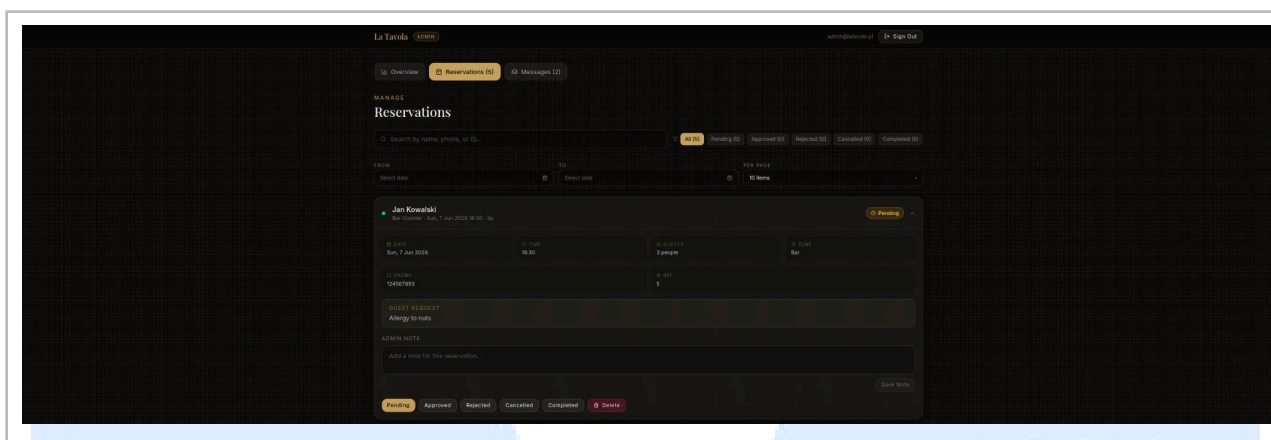
Widok listy wszystkich rezerwacji w panelu admina.



7.7 Panel administracyjny — zarządzanie

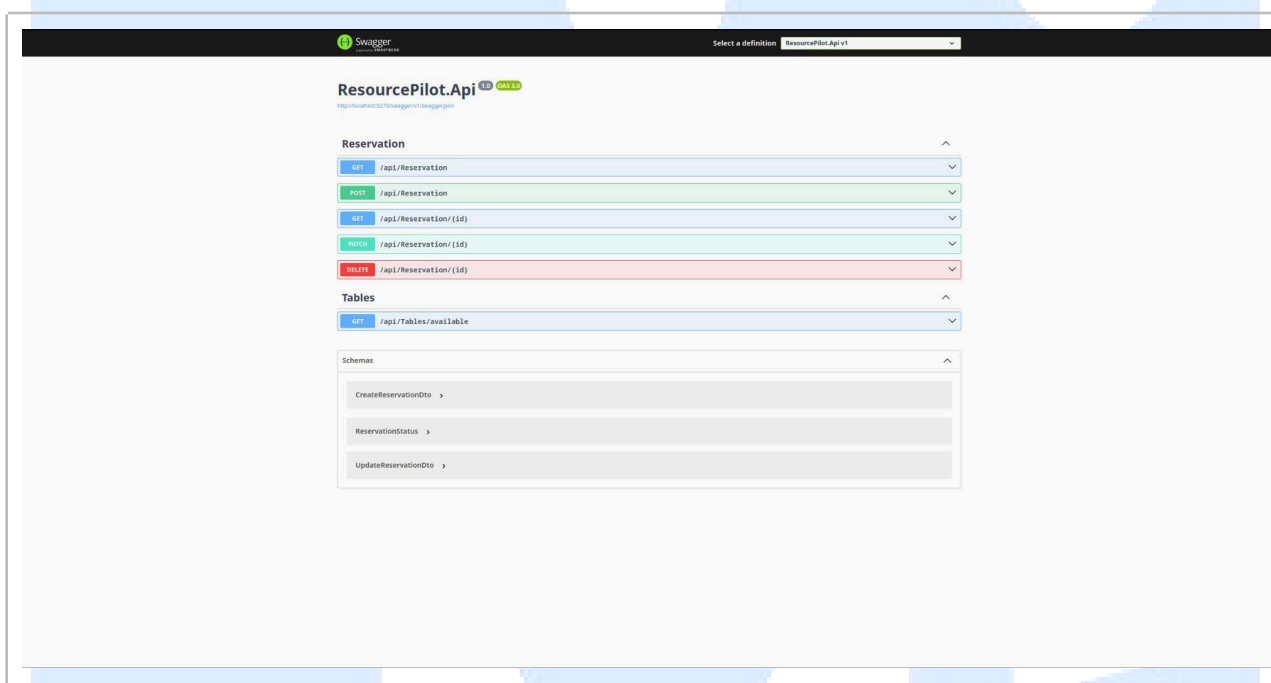
Zmiana statusu rezerwacji przez administratora.

ResourcePilot_dokumentacja



7.8 Dokumentacja Swagger

Interaktywna dokumentacja REST API dostępna pod /swagger.



8. Podział pracy w zespole

Członek zespołu	Rola	Zakres zadań
Marcin Harazmus	Frontend Developer	Implementacja UI w React/TypeScript, komponenty BookingFlow, AdminPanel, ReservationsTimeline, Landing Page, integracja z API backendu (reservationApi.ts)
Daniel Kokot	Backend Developer	Projektowanie architektury Clean Architecture, implementacja warstwy Domain i Application (encje, DTOs, interfejsy), kontrolery REST API, serwis rezerwacji
Maciej Hudek	DevOps / DB / QA	Konfiguracja PostgreSQL, migracje EF Core, integracja frontend-backend, testy połączenia, konfiguracja CORS, dokumentacja projektu

Projekt realizowany był z wykorzystaniem systemu kontroli wersji Git oraz platformy GitHub, co umożliwiło równoległą pracę nad wszystkimi komponentami systemu i śledzenie historii zmian.

9. Wnioski

Projekt ResourcePilot został zrealizowany zgodnie z założeniami i spełnia wszystkie wymagane funkcjonalności. W trakcie pracy zespół zdobył praktyczne doświadczenie w zakresie:

- Projektowania aplikacji webowych w architekturze Clean Architecture
- Implementacji REST API w ASP.NET Core z wykorzystaniem Entity Framework Core
- Obsługi relacyjnej bazy danych PostgreSQL
- Budowania reaktywnych interfejsów użytkownika w React z TypeScript
- Integracji frontendu z backendem przez REST API
- Stosowania wzorców projektowych: DI, DTO, Service Layer, Adapter
- Pracy zespołowej z użyciem systemu kontroli wersji Git

Aplikacja stanowi solidną bazę do dalszego rozwoju. Potencjalne rozszerzenia obejmują implementację uwierzytelniania JWT, powiadomienia email oraz wdrożenie konteneryzacji przez Docker.